

□ AI CRM Brain — Complete Engineering Blueprint

Requirement 1 — Recommendation System Strategy

What Type to Build

You should build a **Hybrid Recommendation System** combining three complementary approaches rather than relying on any single technique. Here's why each layer matters:

Layer 1 — Rule-Based / Business Logic Engine The foundation. Deterministic rules derived from your domain knowledge (e.g., "if a lead has been silent for 7+ days AND the deal stage is 'Proposal', always recommend a direct phone call"). Fast, explainable, and immediately trustworthy for sales teams.

Layer 2 — ML-Based Behavioral Scoring Engine A classification/regression model that learns from historical interaction patterns and deal outcomes to score the *urgency* and *type* of intervention needed per deal.

Layer 3 — LLM-Powered Narrative Engine Takes the structured outputs of Layers 1 and 2 and translates them into human-readable, contextual recommendations using prompt engineering. This is the "face" of the system — what the sales rep actually reads.

Exact Outputs Delivered to the Business User

Output	Example
Next Best Action (NBA)	"Call Maria Chen directly — she's the decision-maker and has opened your proposal email 3 times in the last 24 hours without replying."
Risk Alert	"⚠ Deal D-1004 has been in 'Discovery Call' stage for 18 days (avg is 6). Risk of stalling: HIGH. Recommend re-engaging immediately."
Offer/Content Recommendation	"Send a case study from the FinTech sector — 78% of similar companies responded positively to this asset at this stage."
Engagement Score	Warm 🔥 (Score: 82/100) — based on response time, open rates, and interaction frequency.
Optimal Contact Time	"Best time to reach C-504: Tuesday–Thursday, 10am–12pm (based on past response patterns)."

Requirement 2 — Client Prioritization Methodology

The Scoring Model Architecture

Client prioritization should be a **composite scoring model** that combines three analytical frameworks into a single Lead Priority Score (0–100):

Framework 1 — RFM Analysis (Recency, Frequency, Monetary) Absolutely include this, especially for existing customers (upsell/cross-sell scenarios). Structure it as:

Dimension	Definition	Data Source
Recency (R)	Days since last meaningful interaction	Activity_Date from Interactions Dataset
Frequency (F)	Number of interactions in last 90 days	Count of rows in Interactions Dataset per Deal
Monetary (M)	Deal value + historical purchase value	Deal_Value_USD from Deals Dataset

Assign a score of 1–5 for each dimension, then combine: $RFM_Score = (R \times 0.3) + (F \times 0.3) + (M \times 0.4)$.

Framework 2 — CLTV (Customer Lifetime Value) Strongly recommended — this is what separates a strategic system from a basic one. Formula:

$$CLTV = (Avg\ Deal\ Value \times Avg\ Purchase\ Frequency \times Avg\ Customer\ Lifespan) \times Gross\ Margin\ \%$$

CLTV should feed a **long-term potential score** that prevents your model from always prioritizing the biggest *current* deal over a smaller deal from a customer likely to buy again and again.

Framework 3 — ML Lead Scoring Model A trained classifier (XGBoost recommended) that predicts the probability of deal closure based on firmographic features (Company_Size, Industry, Decision_Maker_Role, Lead_Source) combined with behavioral signals. The output probability directly feeds into the priority score.

Final Priority Score Formula:

$$\begin{aligned} Priority_Score = & (Closure_Probability \times 0.40) \\ & + (RFM_Score_normalized \times 0.30) \\ & + (CLTV_Score_normalized \times 0.20) \\ & + (Engagement_Score_normalized \times 0.10) \end{aligned}$$

Weights are configurable per business context — expose them in the dashboard as tunable sliders for enterprise buyers.

Requirement 3 — Departmental Focus

Beyond Sales, here are the departments to target and how the AI layer serves each:

Sales (Primary — 50% of product focus) Already covered: deal prediction, lead scoring, NBA recommendations, pipeline forecasting.

Marketing (25% of product focus)

- Identify which lead sources (Organic Search, Referral, Trade Show) produce the highest-converting and highest-CLTV customers → feeds budget allocation recommendations.
- Segment customers by behavior and profile → feeds targeted campaign creation.
- A/B test message effectiveness based on Outcome_Flag data from the Interactions Dataset.
- Output: *"Your 'Referral' leads from the SaaS industry have a 3.2× higher close rate. Recommend increasing referral incentives."*

Customer Success / Account Management (15% of focus)

- Churn prediction: flag existing customers showing disengagement signals before it's too late.
- Upsell/Cross-sell timing: use purchase history and engagement data to identify the optimal moment to propose expansion.
- Output: *"TechFlow Inc. (C-501) hasn't engaged in 45 days since their last purchase. Risk of churn: Medium. Recommend a check-in call."*

Business Intelligence / Executive Layer (10% of focus)

- Automated pipeline health reports delivered to leadership.
- Revenue forecasting for the quarter based on aggregated deal probabilities.
- Win/Loss analysis: what factors most influenced closed-won vs. closed-lost deals this quarter.

Requirement 4 — Datasets & Feature Engineering

Datasets Required

The three datasets from your `dataset_usage.docx` form a solid foundation. Here is the full feature engineering specification for each:

Dataset 1: Deals & Opportunities

Raw Field	Engineered Feature	Notes
Created_Date + Close_Date	deal_cycle_days	Duration of the sales cycle
Current_Stage	stage_encoded (ordinal)	Discovery=1, Proposal=2, Negotiation=3, Contract=4
Deal_Value_USD	deal_value_log	Log-transform to reduce skew
Current_Stage + Created_Date	days_in_current_stage	Compare to historical avg per stage
Status	TARGET: is_closed_won (binary: 1/0)	For closure prediction model
Multiple fields	velocity_score	Deal value / days in pipeline

Dataset 2: Customer & Company

Raw Field	Engineered Feature	Notes
Company_Size	company_size_encoded	Ordinal: 1–50=1, 51–200=2, 201–500=3, 1000+=4
Industry	industry_one_hot	One-hot encoding
Decision_Maker_Role	seniority_level	Mapped: CTO/CXO=High, Director=Mid, Manager=Low
Lead_Source	lead_source_encoded	One-hot or target-encoded
Customer_ID join with Deals	historical_win_rate	% of past deals won per customer
Aggregated from Deals	CLTV_estimate	Computed as described above

Dataset 3: Interactions & Activity Logs

Raw Field	Engineered Feature	Notes
Activity_Type	channel_diversity_score	Number of distinct channels used
Activity_Date	days_since_last_contact	Recency signal
Initiated_By	customer_initiation_ratio	% of interactions initiated by customer (high = strong signal)
Outcome_Flag	positive_outcome_ratio	% of interactions with positive/replied outcome
Duration_Mins	avg_meeting_duration	Avg for calls/meetings per deal
Count of rows	total_touchpoints	Total interaction count
Time between activities	avg_response_lag_days	Engagement speed

Requirement 5 — Model Architecture & Scope

Models to Build: 6 Distinct ML Models

Model 1 — Deal Closure Prediction (Core Model)

- **Type:** Binary Classification
- **Algorithm:** XGBoost (primary) with Logistic Regression as interpretable baseline
- **Input features:** All engineered features from Datasets 1, 2, and 3 (joined on `Deal_ID` and `Customer_ID`)
- **Target variable:** `is_closed_won` (1 = Closed-Won, 0 = Closed-Lost)
- **Output:** Probability score (0.0–1.0) for each open deal
- **Department:** Sales

Model 2 — Lead Scoring / Priority Ranking

- **Type:** Regression (scoring) + optional multi-class classification
- **Algorithm:** XGBoost Regressor / Random Forest
- **Input features:** Firmographic features (Dataset 2) + engagement signals (Dataset 3)
- **Target variable:** Composite `Priority_Score` (0–100) as described in Requirement 2, derived from historical win/loss patterns
- **Output:** Ranked list of all open leads with priority tier (High / Medium / Low)
- **Department:** Sales + Marketing

Model 3 — Pipeline Revenue Forecasting

- **Type:** Regression
- **Algorithm:** LightGBM or a simple aggregation model ($\sum Deal_Value \times Closure_Probability$)
- **Input features:** All open deal closure probabilities from Model 1 + `Deal_Value_USD`
- **Target variable:** Total expected revenue for current quarter
- **Output:** Forecasted revenue with confidence interval
- **Department:** Sales + Executive BI

Model 4 — Churn Risk Prediction (High-Value Add)

- **Type:** Binary Classification
 - **Algorithm:** XGBoost on customer behavioral drift signals
 - **Input features:** Change in interaction frequency over time, `days_since_last_contact`, `deal_value_trend`, CLTV
 - **Target variable:** `is_churned` (1 = customer disengaged/lost within 90 days)
 - **Output:** Churn probability per customer with alert threshold
 - **Department:** Customer Success
-

Requirement 6 — 6-Week Sprint Timeline

Sprint 1 (Week 1) — Discovery & Data Infrastructure

Goals: Understand the environment, establish data pipelines, freeze architecture decisions.

- Define the CRM integration strategy: which CRMs to support first (HubSpot, Zoho, CSV upload fallback)
- Stand up a PostgreSQL database with the schema from the three core datasets
- Build data ingestion pipeline: CRM API connectors (OAuth-based) + CSV/Excel upload parser
- Implement data validation layer: check for nulls, type mismatches, date format consistency
- Write a data quality report script to flag incomplete records before they reach models
- **Deliverable:** Working data pipeline that ingests from at least one CRM (or CSV) and loads into the database

Sprint 2 (Week 2) — Feature Engineering & EDA

Goals: Transform raw data into ML-ready feature sets. Understand the data deeply.

- Implement all feature engineering transformations documented in Requirement 4
- Build exploratory data analysis (EDA) notebooks: distributions, correlations, class balance check for Model 1
- Handle class imbalance in the Closed-Won/Lost dataset (use SMOTE or class weighting if needed)
- Define train/test/validation split strategy — time-based split is critical (never use random split on time-series sales data; train on older deals, test on recent ones)
- **Deliverable:** Cleaned, feature-engineered datasets ready for model training; EDA report with key findings

Sprint 3 (Week 3) — Core Model Development

Goals: Train and evaluate Models 1, 2, and 4 (the three most critical to the product's core value).

- Train Deal Closure Prediction model (Model 1) — XGBoost + baseline Logistic Regression
- Evaluate with: AUC-ROC (primary), Precision-Recall curve (important due to class imbalance), and F1 score
- Train Lead Scoring model (Model 2) — implement the composite scoring formula and validate against historical priority decisions
- Train Engagement Scoring model (Model 4)
- Implement SHAP for Model 1 (Model 6 — the explainability layer)
- **Deliverable:** 3 trained, evaluated models with performance metrics documented; initial SHAP analysis showing top features

Sprint 4 (Week 4) — Recommendation Engine & LLM Layer

Goals: Build the NBA recommendation engine and the LLM translation layer.

- Design the prompt engineering framework for the LLM layer — build template prompts that accept structured model outputs and produce natural language recommendations
- Build the recommendation engine logic: combine Model 1 score + Model 4 engagement score + business rules into an action recommendation
- Train/evaluate Pipeline Forecasting model (Model 3) and Churn model (Model 5)
- Build the API layer (FastAPI) exposing model predictions and recommendations as REST endpoints
- **Deliverable:** Working recommendation engine returning human-readable NBAs via API call; all 6 models trained

Sprint 5 (Week 5) — Dashboard, Testing & Integration

Goals: Build the user-facing dashboard and conduct full system testing.

- Develop the frontend dashboard (React-based) showing: prioritized lead list, closure probability badges, engagement temperature, NBA text, and pipeline forecast widget
- Conduct end-to-end integration testing with real or synthetic CRM data
- Performance testing: ensure prediction latency is under 500ms per deal
- User testing with simulated sales manager persona — gather feedback on recommendation clarity, prioritization logic
- Fix any data pipeline failures, model drift edge cases, or API errors uncovered
- **Deliverable:** Functional, integrated system — user can connect a CRM/upload CSV and receive actionable insights on the dashboard

Sprint 6 (Week 6) — Demo, Documentation & Delivery

Goals: Polish, document, and present the final system.

- Record the 5–10 minute demo video walkthrough
 - Write technical documentation: system architecture, API docs, model cards for each of the 6 models
 - Write the user guide: how to connect a CRM, how to read the dashboard, how to interpret recommendations
 - Prepare the Business Case: model the expected revenue uplift using win rate improvement projections (e.g., "if deal closure rate improves from 22% to 28%, what is the revenue impact on a \$2M pipeline?")
 - Prepare the final pitch presentation covering: problem → solution → demo → market size → competitive advantage
 - **Deliverable:** Complete, demo-ready system; all documentation; final pitch deck
-

Requirement 7 — Tech Stack & Tools

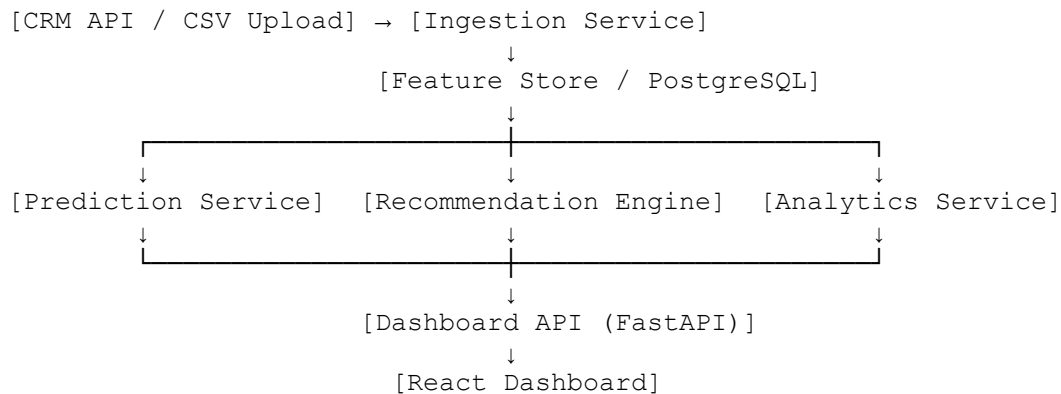
By Phase

Phase	Category	Recommended Tools
Data Ingestion	CRM APIs	HubSpot Python SDK, Zoho CRM REST API, requests library
Data Ingestion	File Upload	pandas (CSV/Excel), openpyxl (Excel), FastAPI file upload endpoint
Data Storage	Database	PostgreSQL (structured data + model results)
Data Storage	ORM	SQLAlchemy
Feature Engineering	Core Libraries	pandas, numpy, scikit-learn (preprocessing pipelines)
ML Models	Training	xgboost, lightgbm, scikit-learn, imbalanced-learn (SMOTE)
ML Models	Explainability	shap
ML Models	Experiment Tracking	MLflow
LLM Layer	API	Anthropic Claude API (claude-sonnet-4) or OpenAI GPT-4o
LLM Layer	Orchestration	LangChain (for prompt templates and chain management)
Backend API	Framework	FastAPI
Backend API	Serving	Uvicorn, Pydantic (data validation)
Frontend Dashboard	Framework	React + TailwindCSS
Frontend Dashboard	Charts	Recharts or Chart.js
Containerization	Docker	Docker + Docker Compose
MLOps	Model Registry	MLflow Model Registry
MLOps	Scheduling	APScheduler or Celery (for periodic model retraining)
Testing	Unit/Integration	pytest, httpx (for FastAPI testing)
Deployment (optional)	Cloud	AWS EC2 / Railway / Render for MVP; AWS SageMaker for enterprise scale

Requirement 8 — Professional Architectural & Business Recommendations

Architecture: Build for Modularity from Day One

Design the system as independently deployable microservices rather than a monolith. This is critical because your go-to-market strategy is selling this as an **add-on to existing CRMs** — different enterprise clients may want only specific modules (e.g., just the deal prediction + dashboard, without the churn model). A microservices design lets you package and price features independently.



Data Privacy & Multi-Tenancy

Every enterprise client will ask: *"Is my data being used to train models for competitors?"* Your architecture must support **strict data isolation** from the start. Use a separate database schema (or separate database) per client tenant. Models should be trained per-client on their own data — do not pool data across clients unless they explicitly opt in. This is both a privacy requirement and a competitive differentiator.

The Cold Start Problem — Have a Plan

New clients will have no historical data to train on. Solve this with a two-phase approach: Phase 1 uses a **pre-trained global model** (trained on public or synthetic sales data) with configurable business rules as the baseline. As the client accumulates 3–6 months of outcome data, Phase 2 triggers **fine-tuning or full retraining** on their own data. Communicate this clearly in the pitch — "The system gets smarter the longer you use it" is a powerful retention argument.

Retraining Cadence

Sales patterns change with market conditions, team behavior, and product offerings. Models should not be static. Implement automated monthly retraining scheduled jobs that pull the latest 12 months of completed deal data, retrain each model, compare performance metrics against the previous version, and only promote the new model to production if it scores better. MLflow handles this versioning automatically.

The Explainability Imperative

Sales managers will not trust (or use) a black-box system. Every prediction on the dashboard must show a simple, plain-language explanation powered by SHAP values. For example, alongside a 78% closure probability score, the dashboard should display: *"Top factors: Deal value is in your typical win range (+), CTO is the decision-maker (+), no response to last 2 emails (-)."* This transparency directly impacts adoption, which is the most common failure mode for AI tools in enterprise sales.

Scalability Path

For the MVP, a single PostgreSQL instance and a FastAPI app running in Docker is perfectly sufficient. When you have enterprise clients with 50,000+ deals in the pipeline, migrate to: a vector database (Pinecone or pgvector) for semantic similarity in recommendations, Apache Kafka for streaming interaction events in real time, and AWS SageMaker or Azure ML for managed model training and endpoint scaling. Document this roadmap in your pitch — enterprise buyers want to know you've thought beyond the demo.

Business Model Alignment

Position the system using a **usage-based SaaS model**: a base subscription fee covering the CRM connection and dashboard, with add-on module pricing for the churn prediction and executive BI layers. This makes the initial sale low-risk for the buyer and creates natural expansion revenue as they adopt more modules. The business case deliverable in Sprint 6 should model this out explicitly — showing a buyer that even a modest improvement in their close rate pays for the annual subscription in the first month is the most powerful thing you can put in front of a decision-maker.

This blueprint gives your engineering team a clear, actionable path from zero to a demo-ready, enterprise-grade system within the 6-week timeline. The foundation you have in `dataset_usage.docx` is solid — your three core datasets map cleanly to the six models and the three-layer recommendation architecture described above. Good luck with the build, Tamer.